

TITLE PAGE

Course Name: Advanced Programming (OOP) Lab

Project Title: Smart Hostel Management System with Database Integration and GUI

Student Name: Richeek Bose

Roll Number: 58

Instructor Name: Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas

Spring 2026

PROBLEM STATEMENT

Hostel management in many educational institutions is still handled manually or through basic tools like spreadsheets and raw file systems. These approaches suffer from several limitations such as lack of data integrity, difficulty in handling concurrent operations, poor scalability, and inefficient data retrieval.

A critical weakness in many simple systems is the reliance on plain file handling, which is not suitable for real-world applications involving multiple users and frequent updates. File-based storage lacks proper structure, security, and concurrency control, making it prone to corruption and inconsistency.

To address these issues, the proposed system introduces a database-driven approach using a lightweight relational database (such as SQLite or H2), ensuring:

- Structured and reliable data storage
- Efficient querying and updates
- Better handling of concurrent operations

Additionally, the system incorporates:

- A Graphical User Interface (GUI) using Java Swing/JavaFX for improved usability
- DAO (Data Access Object) pattern for clean database interaction
- Controlled use of multithreading for responsiveness and background processing

This system aims to provide a robust, scalable, and user-friendly platform for managing hostel operations such as student records, room allocation, complaints, and fee tracking.

SYSTEM DESIGN

UML Class Diagram

<<abstract>>

User

- id

- name

- contact

+ login()

+ displayMenu()

▲

||

Student Admin

Room ----- Student

| roomId | studentId

| capacity | roomId

Complaint

complaintId

studentId

status

DAO Layer:

StudentDAO

RoomDAO

ComplaintDAO

DatabaseManager (Singleton)

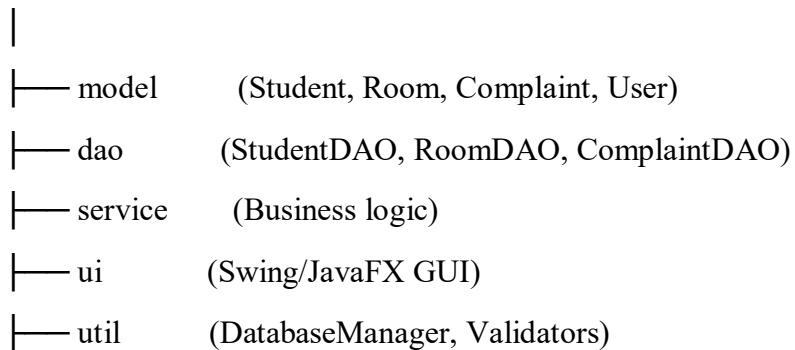
Description of Major Classes

- User (Abstract): Base class defining common attributes and methods
- Student / Admin: Specialized roles with different functionalities

- Room: Manages room details and occupancy
- Complaint: Stores complaint data and status
- DAO Classes: Handle database operations (CRUD)
- DatabaseManager: Singleton class managing DB connection

Package Structure

com.hostelmanagement



Interaction Flow

1. User interacts via GUI
2. UI calls Service Layer
3. Service Layer uses DAO classes
4. DAO interacts with Database
5. Data is returned and displayed in UI

OOP Concepts Demonstrated

1. Abstraction

Where used- Abstract class User

Why- To define a common structure for Admin and Student

Code Snippet

```
Main.java    Share 
```

```
1 abstract class User {  
2     protected int id;  
3     protected String name;  
4  
5     abstract void displayMenu();  
6 }
```

Explanation- Defines a blueprint while forcing subclasses to implement behavior.

2. Inheritance

Where used- Student and Admin extend User

Why- To reuse common attributes and methods

Code Snippet

```
7 class Student extends User {  
8     void displayMenu() {  
9         System.out.println("Student Menu");  
10    }  
11 }
```

Explanation- Student inherits properties from User and overrides behavior.

3. Polymorphism

Where used- displayMenu()

Why- Different behavior for different user types

Code Snippet

```
12 User user = new Student();  
13 user.displayMenu();
```

Explanation- Same method behaves differently at runtime.

4. Encapsulation

Where used- Private variables in models

Why- To protect data integrity

Code Snippet

```
14 class Room {  
15     private int roomId;  
16  
17     public int getRoomId() { return roomId; }  
18     public void setRoomId(int id) { this.roomId = id; }  
19 }
```

Explanation- Restricts direct access to variables.

5. Exception Handling

Where used- Database operations

Why- Prevents crashes

Code Snippet

```

20 * try {
21     Connection con = DriverManager.getConnection(url);
22 * } catch (SQLException e) {
23     System.out.println("DB Error");
24 }

```

Explanation- Handles runtime database failures safely.

6. Collections

Where used- Storing retrieved data

Why- Efficient data handling

Code Snippet

```

25 List<Student> students = new ArrayList<>();

```

Explanation- Stores multiple student objects dynamically.

7. Threads

Where used- Background tasks (GUI responsiveness)

Why- Prevent UI freezing

Code Snippet

```

26 * new Thread(() -> {
27     loadStudents();
28 }).start();

```

Explanation- Runs tasks in background for smooth UI.

IMPLEMENTATION HIGHLIGHTS

1. Database Connection

```
</> Java  
  
Connection con = DriverManager.getConnection("jdbc:sqlite:hostel.db");
```

Justification- Enables structured persistent storage.

2. Add Student via DAO

```
</> Java  
  
PreparedStatement ps = con.prepareStatement(  
    "INSERT INTO student VALUES (?, ?)");
```

Justification- Prevents SQL injection, ensures safety.

3. GUI Button Action

```
</> Java  
  
button.addActionListener(e -> addStudent());
```

Justification- Connects UI to logic

4. Background Loading

```
</> Java  
  
SwingWorker<Void, Void> worker = new SwingWorker<>() {  
    protected Void doInBackground() {  
        loadData();  
        return null;  
    }  
};  
worker.execute();
```

Justification- Keeps UI responsive

TESTING & ERROR HANDLING

Test Cases

- Add student with valid data
- Allocate room
- Submit complaint
- Retrieve records

Edge Cases

- Duplicate student ID
- Full room allocation
- Empty input fields
- Invalid data types

Failure Scenarios

- Database connection failure
- File/database corruption
- Unexpected shutdown

Handled using:

- try-catch
- validation checks
- error messages

CONCLUSION & FUTURE SCOPE

Conclusion

The system successfully demonstrates strong OOP principles along with improved architecture using:

- Database integration
- DAO pattern
- GUI-based interaction
- Controlled multithreading

It is scalable, maintainable, and closer to real-world software systems.

Future Scope

- Web-based version (Spring Boot)
- Mobile app integration
- Role-based authentication system
- Online payment integration
- Advanced analytics dashboard